

RRSI 正则化递归自我改进

让 AI 针对**失败的检查**重写 agent 的系统提示词——Google Research 的做法，以及它防翻车的**四道闸门**

arXiv 2609.24972

github.com/google-research/rrsi

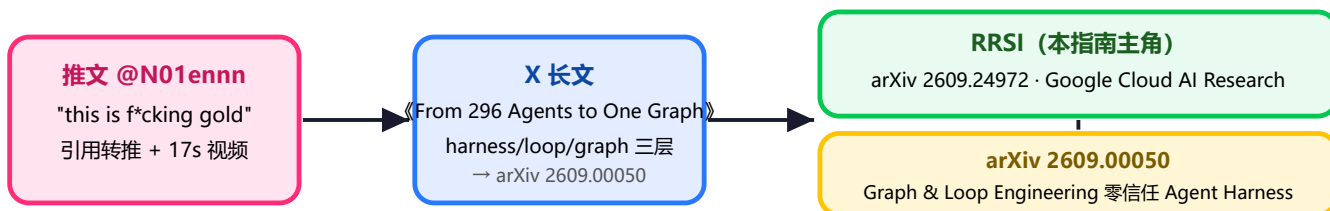
COLM '26

官方单测 8 passed

离线复现实验全跑通

0 | 这条推文说了什么（溯源）

起点是 X 用户 @N01ennn 的高赞转发（7 万+浏览）："Google 工程师讲了如何让 AI **针对失败的检查重写 agent 的系统提示词**——失败检查变成下一个修复任务，回归套件盯着这次修复破坏了什么。" 顺着引用链追到底，本体是两篇 2026 年的工作：



两篇都在 Google 云生态实例化；推文的"失败检查→修复→回归"正是它们共同的内核思想

RRSI = Regularized Recursive Self-Improvement of Agent Harnesses：把 agent 的"挽具"（系统提示词、控制流、工具、记忆、上下文管理）当作可进化对象，让 LLM 在一个评估驱动的循环里**反复改写自己的 harness**。14 位作者，代码开源在 google-research 组织下。

1 | 十二问解答（疑问即目录）

Q1 "让 AI 重写系统提示词"到底是个什么流程？

一句话：把系统提示词当源代码，把评估集当测试套件，跑一个"改提示词→跑测试→只留变强版本"的进化循环。Google 的代码里每轮分两步——**算法 1（提案侧）**：分析当前失败轨迹 → 按预算起草候选 → 泄漏审查；**算法 2（选择侧）**：整套评估 → 四道闸门筛选 → 赢家上岗。落选候选连同分数、理由全部记进 `history.jsonl`，下轮提案时喂回给模型——**被证伪的假设不会被重提**。

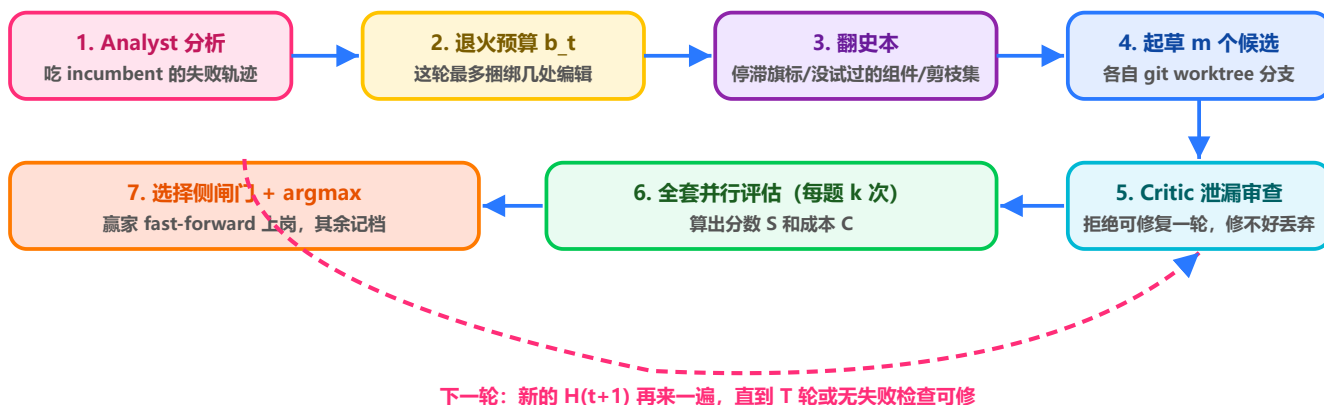
Q2 不搞正则化，直接进化会怎么翻车？

论文的实验发现：无正则化的 RSI 会把训练题背下来——进化集分数大涨（最高 +14.1 分），换一套没见过的题分数就地蒸发。本指南的迷你复现实验（见实验二模式三）完美复刻了这种翻车：写死答案爬到进化集满分，**留出的回归套件得分全程为 0，token 成本反而是诚实版的 2.6 倍**。所以 RRSI 的全部精髓不在“进化”，而在“正则化”——四道闸门（Q5~Q8）。

Q3 什么是 harness？进化它和“调 prompt”差在哪？

harness = 模型外面的一切运行环境（见知识库 [[AgentHarness智能体挽具]]）。RRSI 的可编辑组件集 K 有 9 类：`prompt / control_flow / config / output_plumbing / context_mgmt / client_tool / skill / memory / subagent`。提示词爬山法只调“配方里的一句话”，RRSI 连配方、锅、灶台、进货渠道一起换——但约束只加在“搜索怎么走”上，不限制“挽具最终能长什么样”（开放编辑空间）。

Q4 一轮进化循环里发生哪 7 步？



对应源码 `rrsi/loop.py` 的 `Run.round(t)`，README 有逐条“论文公式→代码位置”映射表。

Q5 编辑预算为什么要“余弦退火”？

公式： $b_t = \text{ceil}(b_{\min} + (b_{\max} - b_{\min}) \cdot \frac{1}{2}(1 + \cos(\pi t/T)))$ 。开局允许一个候选捆绑多处协同编辑（反正离最优远，怎么改都是往上走），临近收尾每轮只许改一处——因为每处编辑都要能单独归因“是它带来涨分”。官方单测第一条就锁这个公式。

实测 `budget_table(T=10, 1..4)` → 4,4,4,4,3,3,3,2,2,2（实验一第 1 段）

Q6 噪声地板 delta——先承认“打分是有抖动的”

同一 harness 跑两次分数都不一样（模型采样有随机性）。**比历史最高 S^* 还低 delta 以内的“涨分”一律当作噪声不算数**： $\text{admissible} \Leftrightarrow S' \geq S^* - \text{delta}$ 。delta 不拍脑袋：用 `calibrate.py` 对基线评估做 bootstrap 自助重采样估计出噪声带。这是防“自我安慰式改进”的第一闸。

Q7 成本规则：涨分必须付得起 token 账

两条分支：**真涨分** ($\Delta S > \text{delta}$) 时，成本涨幅要 $\leq$ 预算 $\beta_0 + \beta_1 \cdot \Delta S$ ——涨得多才许花得多；**带内微涨**时改用整形分 $w_s \cdot \Delta S - w_c \cdot \Delta C + w_n \cdot v > 0$ ，其中 v =触及“从没被采纳过的结构组件”的数量，**只作带内平局打破，鼓励探索新机制但不许抢真涨分的位**。实测 E 候选 $\Delta S = +0.20$ 但 $\Delta C = +8.60 >$ 预算 8.10，被一票否决（实验一第 2 段）。最终版本比无正则进化省 30% 策略 token。

Q8 Critic 六条拒绝规则都在防什么？

Critic 在**评估花钱之前**读 diff，专抓“作弊/退化/自杀式”编辑：①**泄漏/任务特化**——diff 里出现任务名、实体名、任务专属魔数（试金石：这条改动放到另一套题上还成立吗？）；②**退化**——声称加了机制其实是死代码，或删除了安全机制没补；③**评分器作弊**——偷读判分逻辑、硬编码阈值；④**未申报捆绑**——夹带申报清单外的私货；⑤**运行时记忆泄漏**——跨 trial 持久化任务专属数据（这是背题不是改进）；⑥**无界工作**——没有退出条件的“继续改进”循环。拒绝意见回灌提案者修一轮（bounded repair），**修不好就丢弃且不许再提**。

Q9 组件标签怎么防“谎报军情”？

提案者要给每条编辑申报改的是哪类组件，但**申报不算数**，diff 里必须有证据：`text_only()` 发现所有改动行都是字符串字面量/注释 → 一律判 `prompt`；结构性组件 (tool/skill/memory/subagent) 各有正则信号（如 `skills/`、`register_tool`、`.remember()`）。**把控制流改动谎报成“探索新组件”骗探索槽？重标回真实类型，探索槽作废**。实测四种谎报全部被纠正（实验一第 3 段）。

Q10 候选为什么放进 git worktree？

每个候选在 `evolve/<domain>` 分支上开自己的 worktree 和分支起草，审查、评估、提交都在自己树上进行；采纳就 fast-forward，**现任 harness 永远是一个 commit**——出问题可回滚、可 diff、可审计。这正是知识库 [[GitWorktree并行隔离]] 思想的教科书用法。

Q11 每条编辑的"功劳"怎么算？卡住了怎么办？

归因：提案者要预言每条编辑影响哪些题，事后对照逐题分数算 `predicted_hit` 命中率，还要列出"没预料到的回归题"——连月累积成提案者的"信用分"。**停滞检测：**近 w 轮涨幅 $\leq \delta$ 就拉响停滞旗标，下轮强制留出"reserved 探索槽"去碰从没试过的组件。**剪枝：**近期收益为负的机制，连着它在任的代码一起交还提案者移除。推文说的"回归套件盯着这次修复破坏了什么"在这层落到代码：每轮记录 `unpredicted_regressions`。

Q12 引用链另一半：graph / loop / harness 三层工程是什么？

arXiv 2609.00050（推文转发的 X 长文的原型论文）把 agent 工程分三层：**graph engineering** 管长流程怎么走（带验证门控的步骤转移）、**loop engineering** 管失败了怎么爬起来（有界的诊断→修复→重试→复验）、**harness engineering** 管零信任执行（身份、授权、隔离、运行时护栏）。**RRSI 是第四维：让 harness 自己变强的"元循环"**——三层是造车，RRSI 是让车自己升级（见知识库 [[图环挽具三层工程]]）。

2 | 这套东西给你什么能力

能力	你能拿它干什么
读懂自进化 agent 论文	RRSI / RSI-Master / Recuris 这一支 2026 热点的公共词汇量（harness、进化集、OOD、正则化闸门）全部齐备
给自己的 prompt 做回归	最小可行版：一组固定检查题 + 留出题 + "改前改后都跑"——推文说的门槛其实一晚就能搭
搭迷你进化环	实验二的 200 行 toy 就是骨架：失败检查→修复→回归监视→闸门，换上真 LLM 提案者即可升级
识破"刷榜式"agent 宣传	看到"自动优化 prompt 提升巨大"，先问：OOD 留出集多少？成本账怎么算？有没有泄漏审查？
复用 Google 的工程件	退火预算/噪声地板/成本规则/组件打标全是纯函数，Apache-2.0，可以拆出来单独用

3 | 动手实验（全部真实运行，机器：Windows 11 + Python 3.14.7）

实验一 · 用 Google 的真代码跑三段演示

- 1** 克隆并安装 (目的: 拿到官方源码) → 实现: 仓库落盘 → 验证: `git log` 见 `be50316`。
`git clone --depth 1 https://github.com/google-research/rrsi repo`
`pip install -e "path/to/repo[dev]"` (装出 `rrsi-0.1.0` + `pytest`)
- 2** 跑官方单测 (目的: 确认方法核心离线可验) → `python -m pytest tests` → **8 passed in 0.11s**。注意: 测的全是"方法核" (退火/校准/选择/史本/打标), 一个 benchmark 都不碰。
- 3** 运行三段演示 (目的: 亲眼看四道闸门工作) → `python rrsi_core_demo.py` (脚本在 `exercise\`, 只 import 官方包、不改一行 repo 代码)。

✅ 预期结果: 退火表 4→2 递减收 1 | 五候选裁决 A 中选、B 走整形分支、C 触地板、D 被 Critic 拦、E 超成本预算被拒 | 四种谎报标签全部被纠正 | 末行"全部断言通过"

实验二 · toy_rrsi.py: 200 行迷你进化环, 把推文三句话跑给你看

7 道进化题 (看得见的失败检查) + 4 道留出组合题 (回归套件)。harness=系统提示词规则; 策略缺机制就答错。

- 1** 诚实进化: `python toy_rrsi.py` → ✅ 预期: t=0 采纳"打折规则" (`S` 0.286→0.571), t=1 采纳"优惠券规则" (`S_id` 与 `S_ood` 双双 →1.000), t=2 报告"没有可修的失败检查, 进化收敛", 成本 `C`=36。失败的检查真的变成了下一个修复任务。
- 2** 放作弊者进来: `python toy_rrsi.py --proposer leaky` → 提案"若题目编号 E3 (原文...) 直接输出 14.4" → ✅ 预期: 连续 3 轮被 Critic 正则拒绝 (命中两条泄漏规则), bounded repair 终止, 分数停在基线。作弊者一无所获。
- 3** 拆掉 Critic 看翻车: `python toy_rrsi.py --proposer leaky --critic off` → ✅ 预期: 逐题写死答案, 5 轮爬到 `S_id`=1.000——但 `S_ood` 全程 0.000, 成本 `C`=95 (诚实版 2.6 倍): 进化集满分、分布外归零, 这就是论文要防的过拟合。

模式	进化集 <code>S_id</code>	留出套件 <code>S_ood</code>	成本 <code>C</code>	结论
诚实 + Critic 开	1.000	1.000	36	通用机制, 两头都赢
作弊 + Critic 开	0.286 (基线)	0.000	10	闸门拦下, 零损失
作弊 + Critic 关	1.000	0.000	95	背题翻车: ID 满 OOD 空

诚实声明: 官方三个 benchmark 实例 (Terminal-Bench 2.1 / Harvey LAB / EngDesign) 需要 Vertex AI 凭据与 Docker/harbor, 本机未运行; 本指南的结论以源码精读 + 官方单测 + 上述离线复现为限。

4 | 一句话带走 + 三个坑

把系统提示词当代码：有测试（评估集）、有回归套件（留出集）、有 code review (Critic)、有性能预算（成本规则）、有 git (worktree 候选) ——“提示词工程”到这一步正式成为“提示词软件工程”。

坑1 • 只看进化集分数

没有留出回归套件的“自动优化”都是在鼓励背题。
任何 prompt 自动优化先建留出集。

坑2 • 分数抖动当真涨

不校准噪声带 delta 就接受“+0.01”，进化环会围绕噪声原地打转甚至错误采纳。

坑3 • 大胆捆绑式重写

一次改十处的“大优化”无法归因也无法回滚——
退火预算的本质是可归因性。

本指南由拾遗学习流水线产出（任务 8fbb64d3）· 信源：x.com/N01ennn/status/2099530235154739389 → arXiv 2609.24972 (RRSI) + arXiv 2609.00050 (Graph & Loop Engineering) · 代码 github.com/google-research/rrsi (Apache-2.0, HEAD be50316) · 实验材料在同目录 exercise\ · 知识沉淀见 vault：[[RRSI正则化递归自我改进]] [[Critic泄漏审查]] [[图环挽具三层工程]]